

linear search

```
import time

def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

key = int(input("Enter the element to search: "))

start_time = time.perf_counter()

result = linear_search(arr, key)

end_time = time.perf_counter()

if result != -1:
    print(f"\nElement found at index {result}")
else:
    print("\nElement not found")

print(f"Execution Time: {end_time - start_time:.10f} seconds")

print("\nTime Complexity:")
print("Best Case   : O(1)")
print("Average Case: O(n)")
print("Worst Case  : O(n)")
```

```
Enter the number of elements: 6
Enter the elements:
2
9
2
4
8
3
Enter the element to search: 4

Element found at index 3
Execution Time: 0.0000733700 seconds

Time Complexity:
Best Case   : O(1)
Average Case: O(n)
Worst Case  : O(n)
```

binary search

```
import time

def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1
```

```
n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements in sorted order:")
for i in range(n):
    arr.append(int(input()))

key = int(input("Enter the element to search: "))

start_time = time.perf_counter()

result = binary_search(arr, key)

end_time = time.perf_counter()

if result != -1:
    print(f"\nElement found at index {result}")
else:
    print("\nElement not found")

print(f"Execution Time: {end_time - start_time:.10f} seconds")

print("\nTime Complexity:")
print("Best Case   : O(1)")
print("Average Case: O(log n)")
print("Worst Case   : O(log n)")
```

```
Enter the number of elements: 9
Enter the elements in sorted order:
2
4
5
8
1
4
6
9
3
Enter the element to search: 4

Element found at index 5
Execution Time: 0.0000754800 seconds

Time Complexity:
Best Case   : O(1)
Average Case: O(log n)
Worst Case   : O(log n)
```